

Voya: A Zero-Dependency, State-Machine-Driven Architecture for Native SwiftUI iOS Applications

Kaustubha Venkata Eluri
Northeastern University

Abstract—Voya is a native, dependency-free iOS travel-companion application implementing stay discovery, budget planning, travel squads, forkable playbooks, ephemeral stories, and a points-based rewards system across 11 feature modules. This paper documents Voya’s engineering as a craft exercise in disciplined mobile architecture: a strict Model-View-ViewModel (MVVM) separation, a single asynchronous state shape (`ViewState<T>`) that makes invalid UI states unrepresentable at the type level, a protocol-based mock-first service layer enabling parallel frontend/backend development, and a Wilson-score approximation used for both trending-stay and featured-playbook ranking. The application comprises 100 Swift source files and approximately 14,000 lines of code with zero third-party dependencies. We describe the architecture, the central `ViewState<T>` contribution, the ranking algorithm’s mathematical rationale, and the Keychain-based security design, and we close with an honest accounting of the system’s documented limitations and future work.

Index Terms—SwiftUI, MVVM, mobile software architecture, state management, iOS, dependency-free design

I. INTRODUCTION

Modern iOS application development is dominated by third-party dependency graphs: networking, image caching, and reactive state libraries (Alamofire, Kingfisher, RxSwift, and similar) are treated as defaults rather than choices. Voya was built as a deliberate counter-exercise: a full-featured, native SwiftUI travel-companion app — covering stay discovery, per-category budget planning, travel squads, expert-playbook forking, 24-hour travel stories, and a tiered rewards program — implemented with zero external dependencies, on Apple frameworks and the Swift standard library alone.

The motivation is twofold. First, removing third-party dependencies eliminates transitive CVE surface area, reduces binary size, and removes ‘Package.resolved’ churn, at the cost of re-implementing conveniences (image caching, input debouncing, search ranking) that libraries would otherwise provide. Second, and more central to this paper, a dependency-free codebase forces architectural discipline: without a state-management library to lean on, the project adopts a single, explicit asynchronous state shape — `ViewState<T>` — used uniformly across every screen, and a protocol-based service boundary that keeps the app fully testable and previewable without a backend. This paper documents that architecture, the mock-first service design it enables, the Wilson-score-approximation ranking algorithm used for two independent feature domains (trending stays and featured playbooks), and the Keychain-based threat-model reasoning behind Voya’s auth-token storage.

All claims in this paper are drawn directly from the project’s own documentation (`README.md` and `docs/architecture.md`, `docs/decisions.md`, `docs/limitations.md`, `docs/roadmap.md`); no numbers or design claims beyond what is documented are asserted.

II. RELATED WORK

MVVM (Model-View-ViewModel) is a long-established pattern for separating presentation from business logic, and its combination with Apple’s SwiftUI declarative UI framework and the Combine/async-await concurrency models is now a common architectural baseline in native iOS engineering. A recurring theme in reactive-UI state management, across frameworks, is the tension between representing asynchronous screen state as a set of independent boolean/optional flags (e.g., a loading flag, an optional error string, and an optional data payload held separately) versus representing it as a single, mutually exclusive state machine (e.g., a sum type or enum with associated values). The boolean-flag approach is simple to introduce incrementally but admits state combinations that have no sensible UI interpretation — for instance, a screen that is simultaneously loading, displaying an error, and holding stale data. The state-machine approach, by contrast, encodes the screen’s state space as an exhaustive set of mutually exclusive cases, so that a component can never observe a combination that the type system does not allow. This general tension — flags versus algebraic/sum-type state — recurs across reactive frontend ecosystems and is the conceptual backdrop against which Voya’s central design decision, `ViewState<T>`, should be read; Voya’s contribution here is not a novel idea but a disciplined, project-wide application of a single sum-type shape to every asynchronous screen, with no boolean-flag escape hatches, verified as an enforced “quality gate” rather than an aspiration.

Separately, protocol-oriented and dependency-inversion patterns for testability — defining an interface at a service boundary and supplying separate production and mock implementations — are common practice in object-oriented and Swift-specific software design, particularly to enable SwiftUI Previews and unit tests to run without live network access.

III. SYSTEM ARCHITECTURE

Voya is a single-target SwiftUI iOS application (iOS 17.0 minimum target) organized into four layers, enforced project-wide:

- **Views (SwiftUI)** — presentation only. No `URLSession`, no `Codable`, and no business-logic decisions are permitted in this layer; a view that needs data asks its `ViewModel`, and a view that needs to mutate state calls an intent method on its `ViewModel`.
- **ViewModels** — `@MainActor ObservableObject` instances, one per screen (e.g., `HomeViewModel`, `BudgetViewModel`, `SquadViewModel`, `PlaybookViewModel`, `ProfileViewModel`, `StoriesViewModel`, `AuthViewModel`, `DetailViewModel`, `FavoritesViewModel`, `BookingViewModel`, plus `TripsViewModel` and `AppStateManager`), holding orchestration state.
- **Services** — side-effecting components: `StayService`, `AuthService`, `NetworkService`, `CacheService`, `KeychainService`, `BiometricService`, `ImageCacheService`, `SearchRankingService`, `PriceAnalysisService`, `CostOfLivingService`, `LocationService`, `SeasonalService`, and a set of `@MainActor` store singletons (`SquadStore`, `BudgetStore`, `PlaybookStore`, `PointsStore`, `BookingStore`, `StoryStore`, `RecentlyViewedService`), plus `SessionManager` and `HapticService`.
- **Models** — `Codable` value types: `Stay`, `StayCategory`, `User`, `TravelBudget`, `Booking`, `Playbook`, `TravelSquad`, `TravelStory`, `TravelPoints`, `SeasonalRecommendation`, `Review`, `FilterOptions`, and `MockData`.

Figures 1 and 2 show representative client screens rendered by this SwiftUI view layer: the Explore/stay-discovery home screen, driven by `HomeViewModel` and its `ViewState<[Stay]>`-backed `filteredStays` pipeline described below, and the Playbooks feature, driven by `PlaybookViewModel` and `PlaybookStore`.

The codebase comprises 100 Swift source files, approximately 14,000 lines of code, spanning 11 feature modules, with zero third-party dependencies. A single `SessionManager` instance is instantiated app-wide (no duplicate field/init construction), and a documented set of “quality gates” is enforced project-wide, including: a clean `xcodebuild` build against the iOS 17 simulator SDK; no `Color(red:green:blue:)` literals outside `Utils/AppTheme.swift`; no token storage in `UserDefaults` (Keychain only); strict App Transport Security (ATS); portrait-only orientation; universal use of `async/await` except for the one documented Combine debounce; and universal use of `ViewState<T>` for asynchronous screen state, with no scattered loading booleans.

A. The `ViewState<T>` pattern: the paper’s central technical contribution

Every screen with asynchronous data in Voya flows through exactly one enum, defined once and reused across all `ViewModels`:

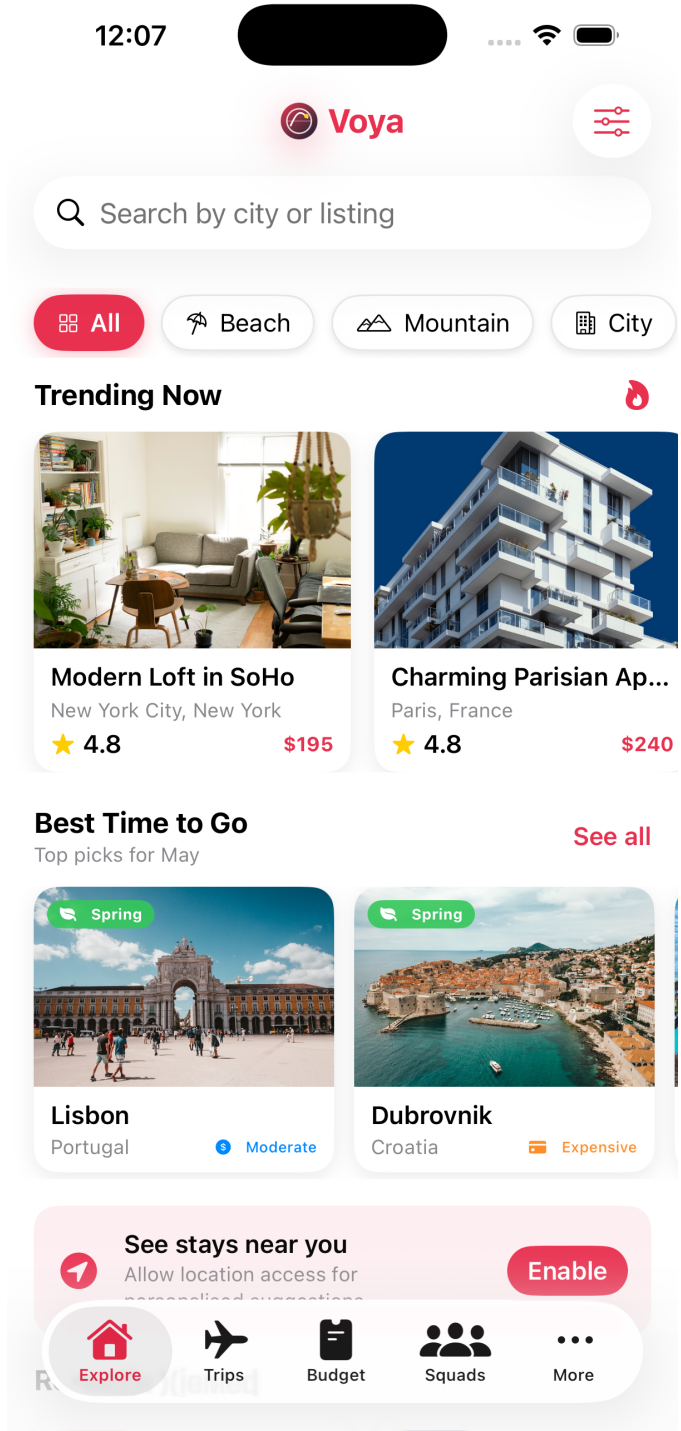


Fig. 1. The Explore screen, Voya’s stay-discovery home view. Content is rendered from `HomeViewModel.filteredStays`, a computed property derived from the single `ViewState<[Stay]>` payload.

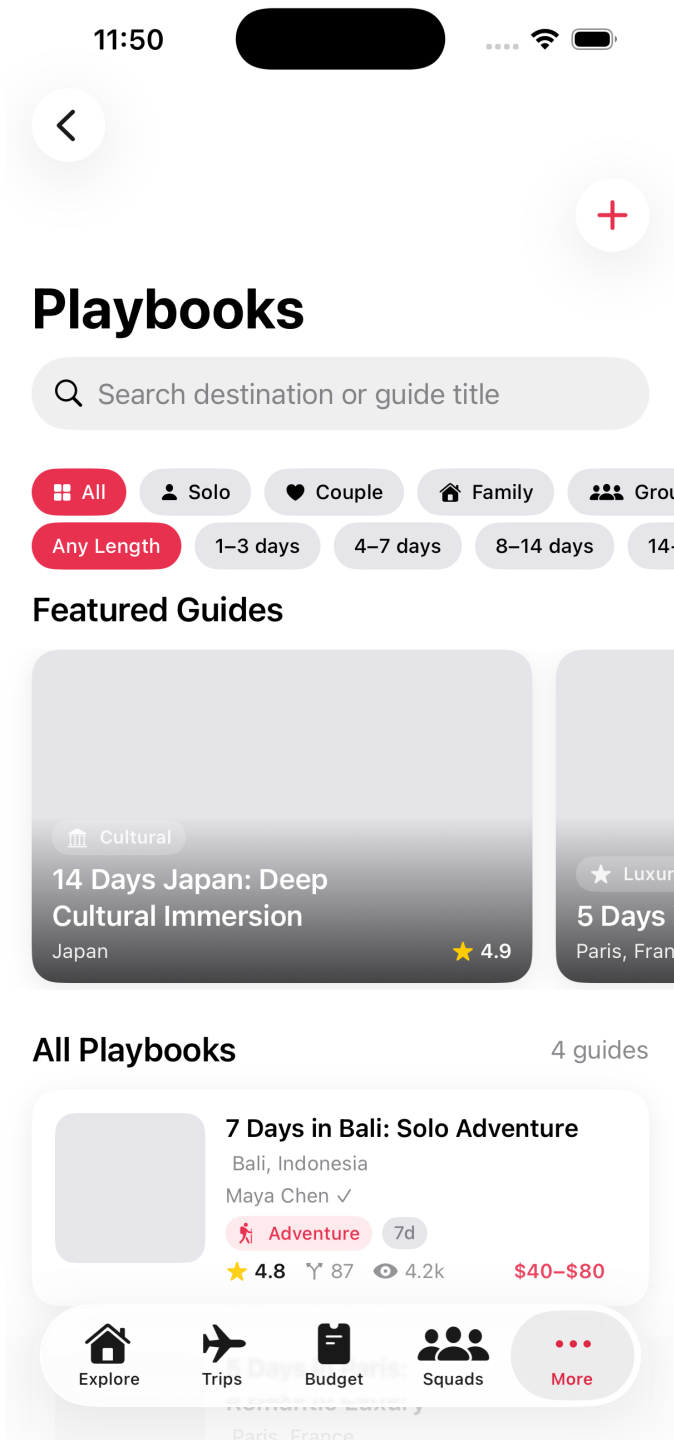


Fig. 2. The Playbooks screen, a distinctive Voya feature for curated, shareable trip itineraries, backed by `PlaybookViewModel` and `PlaybookStore`.

```
enum ViewState<T> {
    case idle
    case loading
    case success(T)
    case failure(String)

    var isLoading: Bool {
        if case .loading = self { true } else {
            false }
    }
}
```

This is documented as Architecture Decision Record ADR-001 (accepted, 2026-05). The design directly targets a well-known anti-pattern in reactive UI state management: representing async screen state as three independent, orthogonally mutable fields — typically `isLoading: Bool`, `error: String?`, and `data: T?`. That representation is combinatorially larger than the set of states that are ever meaningful: it admits `isLoading == true` simultaneously with a non-nil error and a non-nil data, a combination with no coherent UI interpretation, yet fully representable and therefore a latent source of bugs (stale data rendered under a spinner, an error banner shown alongside a success list, etc.). Because the three fields are independently settable, nothing in the type system prevents a `ViewModel` from entering such a state through an incomplete state-transition path (e.g., an early-return bug that sets `isLoading = false` without clearing a previous error).

`ViewState<T>` closes this gap by making the four meaningful states — not-yet-requested (`idle`), in-flight (`loading`), succeeded with a payload (`success(T)`), and failed with a message (`failure(String)`) — the *only* representable states, as mutually exclusive cases of a single enum. Because `HomeViewModel` (and every other screen `ViewModel`) holds `@Published private(set) var state: ViewState<[Stay]>` as its *only* stored async field, there is no second field to fall out of sync: Swift’s compiler-enforced `switch` exhaustiveness check means every consumer of the state must handle all four cases, and adding a fifth case in the future would force every call site to be revisited at compile time rather than fail silently at runtime.

Everything the user sees beyond the raw payload is computed, not stored, eliminating the possibility of a stale duplicate of derived data. Concretely, `HomeViewModel` exposes computed properties such as `filteredStays` (category, filter, and ranked search applied), `trendingStays` (Wilson-score approximation, see Section IV), `recentlyViewedStays`, `nearbyStays` (via `LocationService`, bounded to 2,000 km), `isFiltering`, and `errorMessage` — each an $O(n)$ or $O(n \log n)$ derivation over the single `success([Stay])` payload, recomputed lazily by SwiftUI on `@Published` change. The documented cost of this design is that every consumer of state must perform a `switch` or `if case .success(let value)` unwrap rather than reading a plain optional; this is mitigated by exposing pre-unwrapped computed properties on the `ViewModel` so that the Views layer never touches the enum directly, preserving the rule that

views contain presentation logic only.

The search input pipeline illustrates the pattern end-to-end: user keystrokes populate `searchInput` (`@Published`), pass through a 300 ms Combine debounce into `debouncedSearch` (`@Published`), which triggers recomputation of `filteredStays`, which SwiftUI re-renders. This debounce is documented (ADR-002) as the *only* use of Combine in the entire codebase, present in exactly three files (`HomeViewModel`, `FavoritesViewModel`, `StoriesViewModel`); every other asynchronous operation in the project uses `async/await` and structured concurrency. This is a deliberate, documented mixed-paradigm choice: Combine is retained only where it offers a genuine one-line advantage (stream debounce), and each Combine call site carries a comment explaining the choice.

IV. PROTOCOL-BASED MOCK-FIRST SERVICE DESIGN

Every service in Voya that touches an external boundary is defined as a Swift protocol with two concrete conformance shipped side by side: a production implementation and a mock:

```
protocol StayServiceProtocol {
    func fetchStays() async throws -> [Stay]
}

final class StayService: StayServiceProtocol { /*
    network + cache */ }
final class MockStayService: StayServiceProtocol {
    /* Task.sleep + MockData */ }
```

This is documented as ADR-004 (accepted, 2026-05). ViewModels accept the protocol type in their initializer with the production implementation as the default argument, e.g., `HomeViewModel.init(stayService: StayServiceProtocol = StayService())`, so callers that want the mock (tests, SwiftUI Previews) inject it explicitly while ordinary app startup gets production behavior for free. This mirrors the same seam used for `AuthService`: the documented interface, `signIn(email:password:) async throws -> (token, user)`, is already “real” even though the current implementation is a mock backed by simulated `Task.sleep` network latency and an in-memory `Set<String>` of registered emails; the documented claim is that swapping in a live backend is a single-file change to `AuthService` and `StayService`, with no change to the `ViewModel` or `View` layers.

Two properties documented as motivating this design are (1) enabling SwiftUI Previews and unit tests to inject controlled failures and artificial delays without touching the network, and (2) enabling the frontend to be built and iterated against a fully functional mock while a backend does not yet exist — the entire Debug scheme runs against in-app `MockData` with no network or backend required, and the documented Roadmap Phase 1 (backend hookup: Supabase Auth, Postgres with Row-Level Security, real URL decode in `StayService`) is scoped explicitly to not require view-layer changes. The documented cost of the pattern is a small, fixed per-service overhead (one additional type per service); this is characterized as

acceptable relative to the testability and parallel-development benefit gained.

This same protocol-first, mock-first structure additionally applies to `NetworkService` (generic Decodable decode helper), and store-style services — `SquadStore`, `BudgetStore`, `PlaybookStore`, `PointsStore`, `BookingStore`, `RecentlyViewedService`, and `StoryStore` — which persist via `UserDefaults` and `JSONEncoder/JSONDecoder` (ADR-008), each exposing `@Published` collections and `add(_:)/remove(_:)` intents to the view layer without exposing the persistence mechanism, so a future swap to a real backend store is again documented as a one-file-per-store change.

V. RANKING ALGORITHM

Voya uses the same closed-form ranking expression for two independent feature domains: trending stays on the Explore screen, and featured playbooks. Both are documented as a Wilson-score *approximation*, not a true Wilson lower bound:

$$\text{score}(x) = \text{rating}(x) \times \log(\text{count}(x) + 1) \quad (1)$$

where, for a stay, `count` is the review count, and for a playbook, `count` is the view count. In code (`HomeViewModel.trendingStays`):

```
var trendingStays: [Stay] {
    allStays
        .sorted { a, b in
            (a.rating * log(Double(a.reviewCount) + 1)) >
            (b.rating * log(Double(b.reviewCount) + 1))
        }
        .prefix(6).map { $0 }
```

The same expression, applied to `rating` and `viewCount`, ranks featured playbooks. This is documented as ADR-003 (accepted, 2026-05).

Log-dampening rationale. A pure ranking by raw `rating` would let any stay with a perfect 5.0 score from a single review outrank a stay with a 4.5 average across thousands of reviews, which is statistically unjustified with so little evidence. A pure ranking by raw review/view *count* would instead let sheer popularity dominate regardless of quality. The documented design multiplies `rating` by `log(count + 1)` specifically because a logarithm grows unboundedly but at a decreasing rate: it lets popularity increase a score, so that a well-reviewed but under-exposed item is not permanently buried, while damping that influence enough that a moderately-rated item with very high review count can still be beaten by a higher-rated item with fewer reviews. The documented example given is that a 4.9-rating stay with 100 reviews beats a 4.5-rating stay with 5,000 reviews under this formula, but the same 4.5/5,000 stay still beats a 4.9-rating stay with only 3 reviews — i.e., the dampening is strong enough to reward quality over raw volume within a moderate popularity range, but not strong enough to let a single high rating with negligible evidence win outright.

The $+1$ inside the logarithm exists purely to avoid $\log(0)$ when `count` is zero.

The documentation is explicit that this is *not* a true Wilson score lower bound in the statistical sense, which would require treating ratings as a Bernoulli proportion and computing a confidence-interval lower bound from a Bayesian or normal-approximation prior; the three-line formula above is characterized as a “defensible approximation” of the same intuition, chosen for implementation simplicity given that the underlying dataset (`MockData.stays`) is itself synthetic. ADR-003 documents that this decision should be revisited once the dataset is real data, to consider differential equity between brand-new listings and established ones.

Separately, and not part of the Wilson-style ranking, Voya’s text search uses a distinct multi-factor relevance ranker (`SearchRankingService`) that scores stays by weighted match type — exact title match, title-prefix match, title-contains match, exact/contains location match, per-word token-prefix matches against title/location/description, and a popularity boost term ($\text{rating}/5.0 \times 0.3$) — returning only stays with a positive score, sorted descending, over the visible set.

VI. SECURITY DESIGN

Voya’s authentication tokens are persisted to the iOS Keychain via `KeychainService` (a wrapper over Apple’s `Security` framework), using the access class `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`. This decision is documented as ADR-005 (accepted, 2026-05), with an explicit comparison table against the two alternative classes considered:

- **`WhenUnlockedThisDeviceOnly`** (chosen) — readable only while the device is currently unlocked, and never migrates to a new device (e.g., via encrypted backup restore or device-to-device migration).
- **`AfterFirstUnlockThisDeviceOnly`** — readable once per boot, immediately after the first unlock following a device restart, without requiring the device to still be unlocked at read time; also device-bound.
- **`WhenUnlocked`** (without `ThisDeviceOnly`) — iCloud-Keychain-syncable, so the token would follow the user’s Apple ID across devices.

The threat-model reasoning documented for this choice is that Voya is a foreground-only application: it performs no background refresh, no silent push processing, and no APN-triggered background work, so there is no operational need for the token to be readable while the device is locked or across a boot cycle without a fresh unlock. `WhenUnlockedThisDeviceOnly` is documented as “the tightest valid class” given that constraint. The documentation further notes that if the app later grows a background-sync requirement, the class would be relaxed to `AfterFirstUnlockThisDeviceOnly` — the class used by a sibling project, `HuskyMingle iOS`, for exactly that reason — rather than to the iCloud-syncable `WhenUnlocked` class. The documentation is explicit that `WhenUnlocked` without `ThisDeviceOnly` should never be used for this token,

because it would let the token follow the Apple ID onto a new device, defeating the device-binding property that keeps a stolen or restored token from being usable elsewhere.

Complementary, documented security measures include: biometric app-lock via `LocalAuthentication/LAContext` gating cold launch, toggleable in Settings, with a hard fallback to logout on cancellation; strict App Transport Security, with `NSAllowsArbitraryLoads` explicitly set to `false` (ADR-006, replacing a permissive scaffolding default that was removed because it is an App Store review red flag and an unnecessary attack surface given the app currently makes no live HTTP calls); `ITSAppUsesNonExemptEncryption = false` for App-Store-submission readiness; and a documented quality gate that forbids any token storage in `UserDefaults`, restricting token persistence to the Keychain exclusively.

VII. EVALUATION

The project’s documented verification is a set of enforced build- and convention-level “quality gates” rather than a numeric test-coverage figure; no unit-test coverage percentage is documented in the project’s README or docs, and this paper does not assert one. The documented gates, verified in the codebase per the README, are: (1) `xcodebuild build` passes clean against the iOS 17 simulator SDK; (2) no `Color(red:green:blue:)` literals appear outside `Utils/AppTheme.swift`; (3) no auth token is stored in `UserDefaults` (Keychain-backed only); (4) App Transport Security is strict, with no `NSAllowsArbitraryLoads`; (5) the app is portrait-only on iPhone; (6) all asynchronous code paths use `async/await`, with Combine confined to the one documented debounce use case; (7) all asynchronous screens model state via `ViewState<T>`, with no scattered loading booleans; (8) all shared chrome consumes the `.themeCard()` / `.primaryButton()` / `.chipStyle()` modifiers rather than ad hoc styling; and (9) a single `SessionManager` instance exists app-wide, with no duplicate field/init construction.

Beyond these gates, the documented project statistics are: 100 Swift source files, approximately 14,000 lines of code, 11 feature modules, zero third-party dependencies, 40+ city cost-of-living presets, 36 seasonal destination recommendations (mapped across all 12 months and 4 seasons), and 20+ SwiftUI Preview-enabled views. The presence of 20+ preview-enabled views is itself a downstream benefit of the protocol-based mock-first service design described in Section III, since previews depend on being able to inject a mock service without a live backend.

VIII. LIMITATIONS

The project’s own documentation is explicit about the demo boundary between what is implemented against a real (if mock) service seam and what is client-side/device-local only. No real backend exists: `AuthService` is a mock (in-memory registered-email set plus simulated latency, any well-formed

email with an 8+ character password succeeds, duplicate sign-up correctly fails), and `StayService` falls through to `MockData.stays` rather than reaching the documented mock API endpoint. Trips, budgets, squads, playbooks, points, bookings, stories, and recently-viewed data are all persisted via `UserDefaults`, meaning they are device-local and not cross-installable or synced across devices.

There is no realtime layer: squad chat is a local SwiftUI surface with no WebSocket or realtime-database listener, so a sent message only appends to a local in-memory array; travel stories likewise do not propagate to followers and are visible only on the device that created them. There are no push notifications — no APN certificate and no `UNUserNotificationCenter` integration exist, so booking confirmations, squad invites, and story reminders are UI-only events. There is no map view: `LocationService` exposes a current-city lookup and a within-distance nearby-stays query, but the Explore UI presents results as a list rather than a MapKit overlay. Deep linking is unwired — no universal links or custom URL scheme handler exists in `RootView`. The app has not been submitted to TestFlight or the App Store, though `Info.plist` is documented as already App-Store-ready (strict ATS, `ITSAppUsesNonExemptEncryption = false`, all required usage descriptions present); signing, provisioning, and a privacy manifest remain outstanding. Finally, the mock `AuthService` has documented edge-case gaps: no sign-in rate limiting, no email verification (the mock-returned user is always marked verified), a no-op password-reset method, and decorative (non-functional) SSO buttons for Apple and Google in the UI.

IX. CONCLUSION AND FUTURE WORK

Voya demonstrates that a full-featured, multi-module native iOS application can be built with strict architectural discipline and zero third-party dependencies, centered on a single asynchronous state shape, `ViewState<T>`, that eliminates an entire class of invalid-UI-state bugs at the type-system level, and a protocol-based mock-first service boundary that keeps every feature testable, previewable, and backend-swappable without touching the view layer. The same log-dampened ranking formula is reused across two otherwise unrelated feature domains (trending stays, featured playbooks), and the Keychain access-class choice reflects an explicit, documented threat model rather than a default setting.

The project's own roadmap documents a phased path forward: Phase 1 replaces the mock `AuthService` and `StayService` with a real backend (Supabase Auth with email and Sign in with Apple, Postgres with Row-Level Security, and Supabase Storage for photos and story uploads) without changing the protocol surface or the view layer; Phase 2 adds realtime squad chat via `URLSessionWebSocketTask` against a realtime channel, and realtime story sync scoped to followed users; Phase 3 adds push notifications via an APN certificate and `UNUserNotificationCenter`, triggered by booking confirmations, squad invites, story replies, and points

milestones; Phase 4 expands to Android (Jetpack Compose, mirroring the same protocol-based service pattern) and Web (Next.js 15 with tRPC against the same backend), with `AppTheme.swift`'s brand tokens seeding a shared design-token JSON; Phase 5 adds a MapKit overlay on Explore, universal-link and custom-URL-scheme deep linking, and a full VoiceOver/Dynamic Type accessibility audit; and Phase 6 covers App Store submission, including a privacy manifest and App Store Review. The roadmap explicitly scopes a social graph at production scale, payment processing, and a host-side (as opposed to guest-only) surface as out of scope for the current product direction.